

Lab 2 — Explaining a Model (SHAP) · Tonight in 20 Steps

OPIM 5512 · Module 2 — Explainable AI

The one-sentence version: a good score isn't the finish line — tonight you make the model *explain itself*. One partner builds the **global** view (what drives it overall), one builds the **local** view (why one hour), and you merge them into a report — through the same branch → PR → review → merge loop as Lab 1.

Before we start

- [] **GitHub account + GitHub Desktop** signed in · **Colab** opens
- [] You read (or heard) [SHAP_LECTURE.md](#) — beeswarm = global, waterfall = local
- [] Roles picked: **Partner A = global, Partner B = local**
- [] Your only code is **one SHAP line**. Type it.

⚠ The 3 things that trip everyone up (read this first)

1. **Partner B must ACCEPT the email invite before anything works.** No accept = your name won't appear as a reviewer and your pushes get rejected. Check your inbox (and spam) the moment Partner A adds you.
2. **Never commit straight to `main`.** Always work on your `dev-` branch → push → pull request. Pushing to `main` is what makes the network graph loop — skipping it means no loop, no credit.
3. **After you publish a branch, click Fetch/Pull in GitHub Desktop** or the folder looks empty. And **GitHub can lag on lab night** — if the network graph or a PR doesn't show, wait ~30s and refresh.

Flying solo (no partner / online)? You *are* both A and B. Make the repo, set approvals to **0**, and make **two branches** (`dev-global` , `dev-local`) so you still get the branch → PR → merge loop and a network graph. Do the global notebook first, then the local one.

The 20 steps, in order

Set up (steps 1–6)

1. **Pair up:** Partner A (global) / Partner B (local). (*Solo/online? Two accounts, or pair over Teams.*)
2. **Partner A:** open the [template repo](#) → **Use this template** → **Create a new repository** → owner = *you*, name `opim5512-lab2-<netidA>-<netidB>`, **Public** → Create.
3. **Partner A: Settings** → **Collaborators** → **Add people** → Partner B. → **STOP: Partner B opens their email and ACCEPTS the invite now.** Nothing else works until this is done — check spam if it's not there.
4. **Partner A: Settings** → **Rules** → ruleset on `main`: **require a pull request + 1 approval**. (*Solo? approvals = 0.*) This lock is what forces the PR loop — don't skip it.
5. **Both:** GitHub Desktop → **File** → **Clone repository** → pick the repo → **Clone**. Once. (*If the repo isn't listed for Partner B, the invite wasn't accepted — go back to step 3.*)

6. **Each:** **Current branch** → **New branch** → `dev-global` (A) / `dev-local` (B) → **Publish branch**. Then **Fetch/Pull** so your local folder actually has the files (a fresh branch can look empty until you do).

Explain the model (steps 7–12)

7. **Each:** Colab → **File** → **Open notebook** → **GitHub tab** → your repo URL → open `Lab2_A_Global_SHAP.ipynb` (A) / `Lab2_B_Local_SHAP.ipynb` (B).
8. **Each:** **Runtime** → **Run all**. Setup installs SHAP, fits the model (note the R^2), builds `shap_values`, and hands you one free plot.
9. **Each:** in the **TODO** cell, write your **one SHAP line** (shape is printed above it): - **A:**
`shap.plots.beeswarm(shap_values, show=False)` → save `shap_global.png` - **B:** `i =`
`int(np.argmax(model.predict(X)))` then `shap.plots.waterfall(shap_values[i], show=False)` → save
`shap_local.png`
10. **Each:** **look at your plot** and say what it shows. Run the **download** cell → two PNGs → Downloads.
11. **Each:** **File** → **Save** the notebook → *your* repo · *your* `dev-` branch · keep the path · real commit message.
(*Plain Save* — not *Ctrl+S*.)
12. **Each:** GitHub Desktop → **Repository** → **Show in Explorer** → drag both PNGs into `images/` (*rename any*
(1) *first*) → top bar = your `dev-` branch → **Commit** → **Push**.

Review & merge (steps 13–16)

13. **Each:** github.com → **Compare & pull request** → title → **Create** → **Reviewers** → your partner. (*Partner not in the reviewer list? They never accepted the invite — step 3.*)
14. **Each:** open your **partner's** PR → **Files changed** → read their SHAP cell, look at the PNG → **Approve**.
15. **Merge** both PRs → **Delete branch** on each.
16. **Both:** GitHub Desktop → **main** → **Fetch** → **Pull**. `images/` now has **four PNGs** — global + local of the same model.

Report & read-out (steps 17–20)

17. **One screen, two people:** open `REPORT.md` →  **Edit** → one sentence per → line (units on every number).
18. **Commit the report** to a new branch `report` → PR → the *other* partner approves → **Merge**. (*main is protected — the rule working.*)
19. **If you're ahead:** run `Lab2_Joint_Optional.ipynb` → `shap_dependence.png` → `images/` → PR → merge → drops into section 5.
20. **Read-out:** beeswarm + waterfall on screen, one sentence each — **do global and local agree on what drives demand?** Check **Insights** → **Network**.

Don't panic

- **No module named 'shap'?** Run all from the top — cell 1 installs it.
- **SHAP plot errors?** You skipped the setup cell that makes `shap_values`. Run all from the top.
- **Commit early.** Once committed, it's essentially impossible to lose.
- `main` **rejecting your push is the protection working** — switch to your `dev-` branch.
- **Same path every save** — a different path makes a *second* notebook.

- **Reviewer/PR/branch not showing up?** 9 times out of 10 the invite wasn't accepted (step 3). Otherwise GitHub is just lagging — wait ~30s and refresh.
 - **No loop in the network graph?** Someone committed straight to `main`. Make a `dev-` branch, add your name to the README, commit → PR → merge, and the loop appears.
 - **Empty folder after a new branch?** Hit **Fetch origin** then **Pull** in GitHub Desktop.
-

Definition of done — what's in the repo at 7:30

- [] Both partners are collaborators; **branch protection** on `main`
- [] `images/` has **four PNGs** with the exact filenames (two per partner)
- [] Both notebooks saved back with **one SHAP line** each
- [] `REPORT.md` — one real sentence per plot, plus one **honest-limits** line: *SHAP explains the model, not the real world — it shows the pattern the model leaned on, not proof of cause.*
- [] **≥3 merged pull requests** (one each + the report), branches deleted, both authoring **and** reviewing
- [] A **network graph** showing the loop going both ways

Global says what the model leans on. Local tells the story of one hour. Tonight you shipped both — and neither of you could have alone.